



Habib University
shaping futures

CS201 – Data Structures II

Lecture 13

Dr. Muhammad Mobeen Movania

(Content has been taken from the lecture notes of
Abdullah Zafar and Dr. Syeda Saleha Raza)



Outlines

- Motivation
- Terminologies
- Grepping
- Boolean retrieval
- Inverted index
 - Building an inverted index
 - Implementation
- Query optimization
- Preprocessing

Motivation

- Tries could be used to perform single-string search in optimal time
 - They help search for prefixes in time linear to the prefix
- What if we want to perform a compound search (involving more than one string) in a body of text?
 - For e.g. What if we wanted to find all plays by Shakespeare with the following terms:
 - Brutus AND Caesar
 - Brutus AND Caesar BUT NOT Calpurnia
 - Brutus NEAR Caesar
- The field is broadly called information retrieval
 - Information retrieval (IR) is finding material (usually documents) of an unstructured nature (usually text) that satisfies an information need from within large collections (usually stored on computers).

Terminologies

- ***Information need***: The topic about which the user desires to know more.
- ***Query***: What the user conveys to the computer in an attempt to communicate the ***information need***.
- ***Documents***: The units we have decided to build a retrieval system over.
- ***Collection (aka corpus)***: A group of documents over which we perform retrieval.
- ***Relevance***: A document is ***relevant*** if it is one that the user perceives as containing information of value with respect to their personal information need.
- Measures of effectiveness of an IR system
 - ***Precision***: What fraction of the returned results are ***relevant*** to the information need?
 - ***Recall***: What fraction of the ***relevant documents*** in the ***collection*** were returned by the system?

Idea 1: Grepping

- Basic Idea: We linearly search through the text for query
- Issues:
 - Might work for small amount of documents, but to process large document collections quickly it might not work
 - Need for more flexible matching operations.
 - Grepping might not be able to help in queries like “Brutus NEAR Caesar” here NEAR could mean within a few words or within the same sentence
 - Need to have a ranked retrieval
 - To have the best answer from a query

Idea 2: Boolean retrieval

- Basic Idea: We index the documents in advance and generate binary term-document *incidence matrix*.
 - Terms are the *indexed units* which are the unique words we want to search for.

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
Antony	1	1	0	0	0	1	
Brutus	1	1	0	1	0	0	
Caesar	1	1	0	1	1	1	
Calpurnia	0	1	0	0	0	0	
Cleopatra	1	0	0	0	0	0	
mercy	1	0	1	1	1	1	
worser	1	0	1	1	1	0	
...							

► **Figure 1.1** A term-document incidence matrix. Matrix element (t, d) is 1 if the play in column d contains the word in row t , and is 0 otherwise.

Example query and search results

- To answer the query Brutus AND Caesar AND NOT Calpurnia, we take the vectors for Brutus, Caesar and Calpurnia, complement the last, and then do a bitwise AND

Brutus AND Caesar = 110100 AND 110111

1 1 0 1 0 0
& 1 1 0 1 1 1

1 1 0 1 0 0

Search result: Antony & Cleopatra, Julius Caesar and Hamlet.

Calpurnia = 010000

NOT Calpurnia = 101111

1 1 0 1 0 0 Brutus AND Caesar

1 0 1 1 1 1 NOT Calpurnia

1 0 0 1 0 0

Search result: Antony & Cleopatra and Hamlet.

Exercise: Build the term document index for the given documents



- **Doc 1** breakthrough drug for schizophrenia
- **Doc 2** new schizophrenia drug
- **Doc 3** new approach for treatment of schizophrenia
- **Doc 4** new hopes for schizophrenia patients

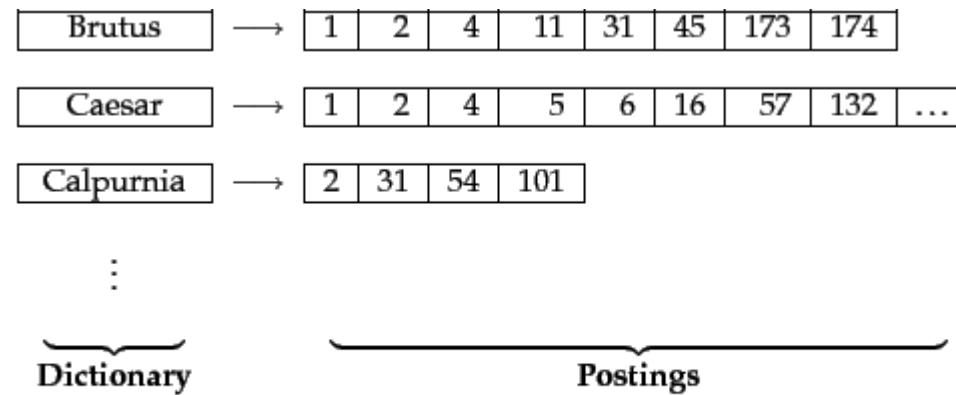


Issues with Idea 2: Boolean retrieval

- Term-Document Matrix can become too big too quickly
 - A 500Kx1M matrix has half-a-trillion 0's and 1's - too many to fit in a computer's memory.
- Term-Document Matrix suffers from information sparseness
 - A small proportion of 1s compared to 0s so we should record only 1s
- Can help with queries like “Brutus AND Caesar” or “Brutus AND Caesar BUT NOT Calpurnia” but cannot help in queries like “Brutus NEAR Caesar”

Idea 3: Inverted Index

- We keep a **dictionary**: which is a collection of terms
- For each term in the dictionary, we have a list (called **posting list** or **postings**) that records which documents the term occurs in.
 - Each item in the list - which records that a term appeared in a document (and, later, often, the positions in the document) - is conventionally called a **posting**



► **Figure 1.2** The two parts of an inverted index. The dictionary is commonly kept in memory, with pointers to each postings list, which is stored on disk.

How do you build an inverted index?

- Four step process:

- Step 1: Collect all the documents to be indexed.

Friends, Romans, countrymen. So let it be with Caesar

- Step 2: Tokenize the text, turning each document into a list of tokens.

Friends Romans countrymen So ...

- Step 3: Generate indexing terms by linguistic preprocessing, producing a list of normalized tokens.

friend roman countrymen so ...

- Step 4: Index the documents that each term occurs in by creating an inverted index, consisting of a dictionary and postings.

Example

- Building a basic inverted index by *sort-based indexing*.

Doc 1

I did enact Julius Caesar: I was killed
i' the Capitol; Brutus killed me.

Doc 2

So let it be with Caesar. The noble Brutus
hath told you Caesar was ambitious:

term	docID	term	docID
I	1	ambitious	2
did	1	be	2
enact	1	brutus	1
julius	1	brutus	2
caesar	1	capitol	1
I	1	caesar	1
was	1	caesar	2
killed	1	caesar	2
i'	1	did	1
the	1	enact	1
capitol	1	hath	1
brutus	1	I	1
killed	1	I	1
me	1	i'	1
so	2	it	2
let	2	julius	1
it	2	killed	1
be	2	killed	1
with	2	let	2
caesar	2	me	1
the	2	noble	1

term	doc. freq.	→	postings lists
ambitious	1	→	2
be	1	→	2
brutus	2	→	1 → 2
capitol	1	→	1
caesar	2	→	1 → 2
did	1	→	1
enact	1	→	1
hath	1	→	2
I	1	→	1
i'	1	→	1
it	1	→	2
julius	1	→	1
killed	1	→	1
let	1	→	2
me	1	→	1
noble	1	→	2

caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2

noble	1	→	2
so	1	→	2
the	2	→	1 → 2
told	1	→	2
you	1	→	2
was	2	→	1 → 2
with	1	→	2



Exercise: Build an inverted index for the given documents

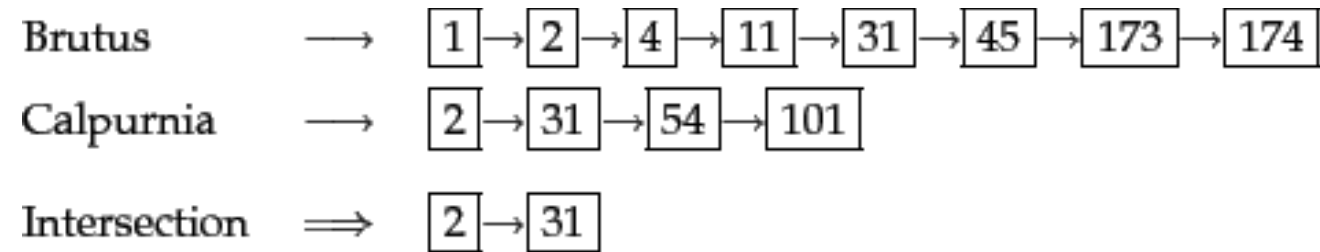
- **Doc 1** breakthrough drug for schizophrenia
- **Doc 2** new schizophrenia drug
- **Doc 3** new approach for treatment of schizophrenia
- **Doc 4** new hopes for schizophrenia patients

Inverted Index: Implementation

- Term Dictionary is kept in main memory, Postings in secondary storage
- Which data structure can be used for terms dictionary?
 - Hash Tables: $O(|\text{string}|)$ expected search complexity
 - Tries: $O(|\text{string}|)$ search complexity
- Which data structure can be used for postings?
 - Arrays - $O(|\text{postings}|)$ search complexity, used when updates are less frequent
 - Linked list - $O(|\text{postings}|)$ search complexity, used when updates are more frequent
 - Skiplist - $O(\log |\text{postings}|)$ expected search complexity

Inverted Index: Processing Boolean Queries

- Suppose we have a simple conjunctive query “Brutus AND Calpurnia” that needs to be processed for the inverted index shown below:



- Steps:
 1. Locate Brutus in the Dictionary
 2. Retrieve its postings
 3. Locate Calpurnia in the Dictionary
 4. Retrieve its postings
 5. Intersect the two postings lists,

Inverted Index: Intersect function

- Time Complexity:
 - $O(|p1| + |p2|)$

```
Intersect(p1, p2)
{
    answer ← {}
    while p1 != NULL and p2 != NULL
    do
    {
        if docID(p1) == docID(p2)
        {
            Add(answer, docID(p1))
            p1 = p1->next
            p2 = p2->next
        }
        else if docID(p1) < docID(p2)
            p1 = p1->next
        else
            p2 = p2->next
    }
    return answer
}
```

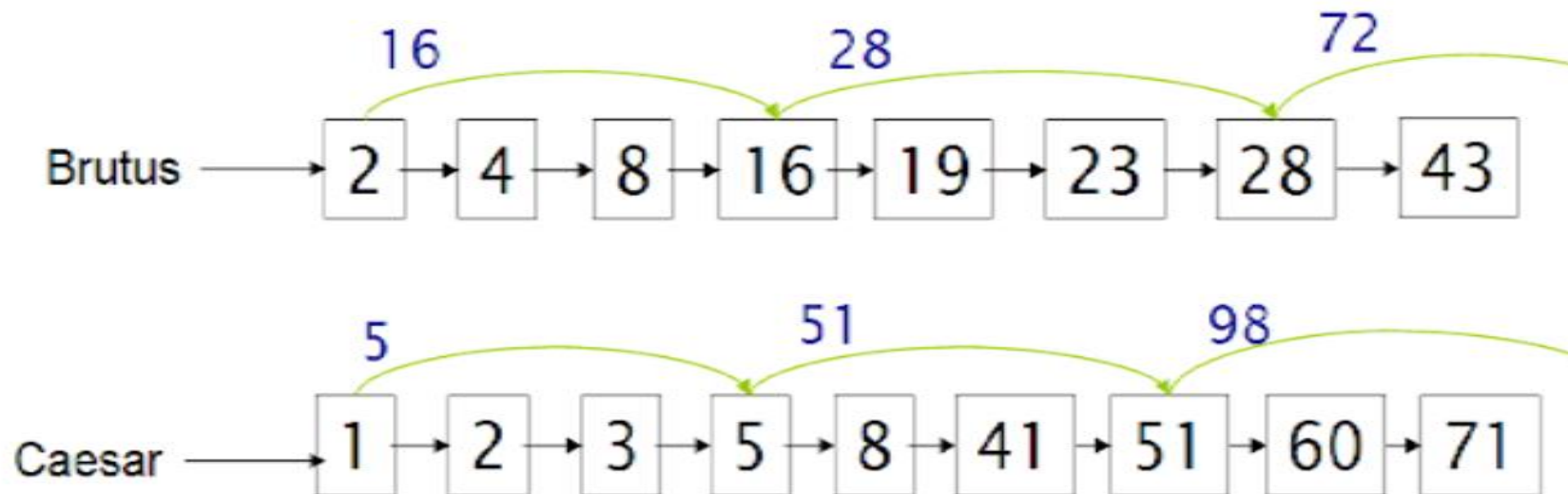

Inverted Index: Union function

- Time Complexity:
 - $O(|p1| + |p2|)$

```
Union(p1, p2)
{
    answer ← {}
    while p1 != NULL and p2 != NULL
    do
    {
        if docID(p1) == docID(p2)
        {
            Add(answer, docID(p1))
            p1 = p1->next
            p2 = p2->next
        }
        else if docID(p1) < docID(p2)
        {
            Add(answer, docID(p1))
            p1 = p1->next
        }
        else
        {
            Add(answer, docID(p2))
            p2 = p2->next
        }
    }
    return answer
}
```

Faster posting list intersection via skip pointers

- Recall that in the intersect function, we iterate over two posting lists simultaneously.
- This gives a runtime complexity of $O(m+n)$ if the first posting list has length m and second has length n .
- Can we process postings list intersections in sublinear time?
 - Yes we can using skiplists that is we augment postings lists with skip pointers (at indexing time).



Query Optimization

- It is the process of selecting how to organize the work of answering a query so that the least total amount of work needs to be done.
- A major element of this for Boolean queries is the order in which postings lists are accessed. What is the best order for query processing?
- Consider the query “Brutus AND Caesar AND Calpurnia”
 - For each of the t terms, we need to get its postings, then AND them together.
 - The standard heuristic is to process terms in order of increasing document frequency
- If we start by intersecting the **two smallest postings lists**, then all intermediate results must be no bigger than the smallest postings list, and we are therefore likely to do the least amount of total work.

Example

- Consider the query “Brutus AND Caesar AND Calpurnia” with postings of size 8, 18 and 4

Brutus	→	1	2	4	11	31	45	173	174	
Caesar	→	1	2	4	5	6	16	57	132	...
Calpurnia	→	2	31	54	101					

- The query can be optimized to (Calpurnia AND Brutus) AND Caesar

Preprocessing

- Removing stopwords

a an and are as at be by for from
has he in is it its of on that the
to was were will with

Figure 2.5: A stop list of 25 semantically non-selective words which are common in Reuters-RCV1.

- Removing hyphens
 - anti-discriminatory and antidiscriminatory
- Abbreviations
 - U.S.A and USA should be same but C.A.T should not be replaced with cat
- Maintaining synonyms
 - Car and automobile,
- Stemming
- Lemmatization

Stemming and Lemmatization

- Goal of both stemming and lemmatization is to reduce inflectional forms and sometimes derivationally related forms of a word to a common base form.
- Example:
 - am, are, is → be
 - car, cars, car's, cars' → car
- *Stemming* usually refers to a crude heuristic process that chops off the ends of words in the hope of achieving this goal correctly most of the time, and often includes the removal of derivational affixes.
 - For e.g. programs, programming, programmer can be reduced down to stem “program”
- *Lemmatization* usually refers to doing things properly with the use of a vocabulary and morphological analysis of words, normally aiming to remove inflectional endings (s, ed, ing forms) only and to return the base or dictionary form of a word, which is known as the *lemma*.
 - For e.g. play, played, playing can be reduced down to lemma “play”
- Be careful though stemming on caring will return “car” but lemmatization will return “care”



Further Reading

- **Introduction to Information Retrieval** by Christopher D. Manning, Prabhakar Raghavan & Hinrich Schütze, Cambridge University Press, 2008. Available online: [Introduction to Information Retrieval \(stanford.edu\)](#)
 - Chapter 1, 2
- Abdullah Zafar's lecture notes on inverted index. Available online: [Week 13 - Inverted Index - Google Docs](#)



Next Lecture

- Lecture 14